



SZABIST
UNIVERSITY

FINAL YEAR PROJECT PROPOSAL

VERIXA

An Agentic AI Quality Engineer

Project Team

Arisha Mumtaz

2312358

Farheen Imam

2312363

Project Supervisor

Asim Riaz

Date of Submission: 18th September 2026

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science in Computer Science

in the

Faculty of Computing and Engineering Sciences

Shaheed Zulfiqar Ali Bhutto Institute of Science and Technology University

(SZABIST University) Karachi Campus

Contents

Contents	2
1. Introduction	3
2. Objective	3
3. Problem Description	3
4. Target Industry	4
5. Methodology	4
5.1 Orchestration	4
5.2 Language Models	6
5.3 State and Data	6
5.4 Integrations	7
5.5 Backend and Dashboard	7
6. Project Scope	7
6.1 In Scope for This Semester	7
6.2 Deferred to Next Semester, Not Dropped	8
6.3 Out of Scope for the Full Year	8
6.4 Assumptions	8
7. Feasibility Study	8
7.1 Risks Involved	8
7.2 Resource Requirement	9
8. Solution Application Areas	9
9. Tools/Technology	10
10. Expertise of the Team Members	11
11. Milestones	11
11.1 FYP-I Features	11
11.2 FYP-II Features	12
12. Project Schedule	14
13. Work Breakdown Structure	14
13.1 Semester 1 (Fall): FYP-1 Work Breakdown	15
13.2 Semester 2 (Spring): FYP-2 Work Breakdown	16
14. References	18

1. Introduction

Verixa is an agentic AI quality engineer. It is a system that watches a pull request the moment it is opened, works out which tests actually matter for that change, runs those tests, gathers the evidence, and drafts a defect ticket that a human can approve before it reaches Jira. Instead of one large program trying to do everything at once, Verixa is built from a small team of specialised agents that pass work to each other, with a manager agent checking every hand off along the way.

Verixa is built to make this workflow better for testers by automating the entire journey from a pull request being opened to a confident release. It uses multiple specialised agents that work together, acting like an AI employee that assists the tester at every step rather than replacing their judgement.

This document describes the seventh semester build of Verixa. Rather than attempting every feature from the original idea at once, the team has deliberately narrowed the seventh semester to one complete and reliable loop, from a pull request being opened to a ticket being filed, with the more advanced features planned for the eighth semester. This proposal explains the objective, the problem being solved, the approach taken, the scope, the feasibility, and the plan for both semesters.

2. Objective

To design and build Verixa, an agentic AI quality engineer that connects to GitHub and Jira, automatically scores the risk of a newly opened pull request, runs the relevant automated tests against a staging environment, collects and organises the failure evidence, and drafts a ready to review defect ticket for a human to approve, using a supervised multi agent system built on Google ADK [1]. Verixa also takes the tester's existing API collections and automation scripts as input, and drafts new test cases for any coverage gaps it finds in what was uploaded.

3. Problem Description

When a developer opens a pull request today, a tester usually has to move between several separate tools to review it properly. One tool is needed to judge how risky the change looks, another to run the relevant tests, another to read through a trace or a log when something fails, another to keep track of manual test cases, and finally Jira itself, where the tester writes up the defect by hand. Each of these tools works well on its own, but none of them carry context to the next one. The tester ends up retyping the same information, screenshot references, error messages, and the reasoning behind a decision, several times over.

This repeated, manual hand off is slow and repetitive, even for experienced testers, and it is exactly the kind of work that benefits from automation without needing to replace human judgement. The real problem is not that good testing tools do not exist. Playwright, Newman, and Jira already do their individual jobs well. The problem is that nothing free currently chains these steps together automatically while still keeping a human in control of the final decision to file a ticket.

Verixa is designed to close that specific gap. It does not attempt to replace Playwright as a test runner or Jira as a tracking tool. It sits on top of them, carrying the risk score, the selected tests, the collected evidence, and the drafted ticket across every step automatically, with one approval gate before anything is filed.

4. Target Industry

Verixa is aimed at the Information Technology and Software Development industry, and specifically at developers and testers . Any software team that reviews pull requests on GitHub and tracks defects in Jira is a potential user, from small startups with a lean QA process to larger engineering teams looking to reduce the manual overhead around test evidence and ticket writing.

5. Methodology

Verixa is built around a supervisor style multi agent architecture rather than one large autonomous agent. This pattern is documented by both Anthropic and OpenAI as the standard way to keep a multi agent system reliable, since a central manager can validate each agent's output before letting the workflow move forward, instead of letting errors compound silently through the pipeline.

5.1 Orchestration

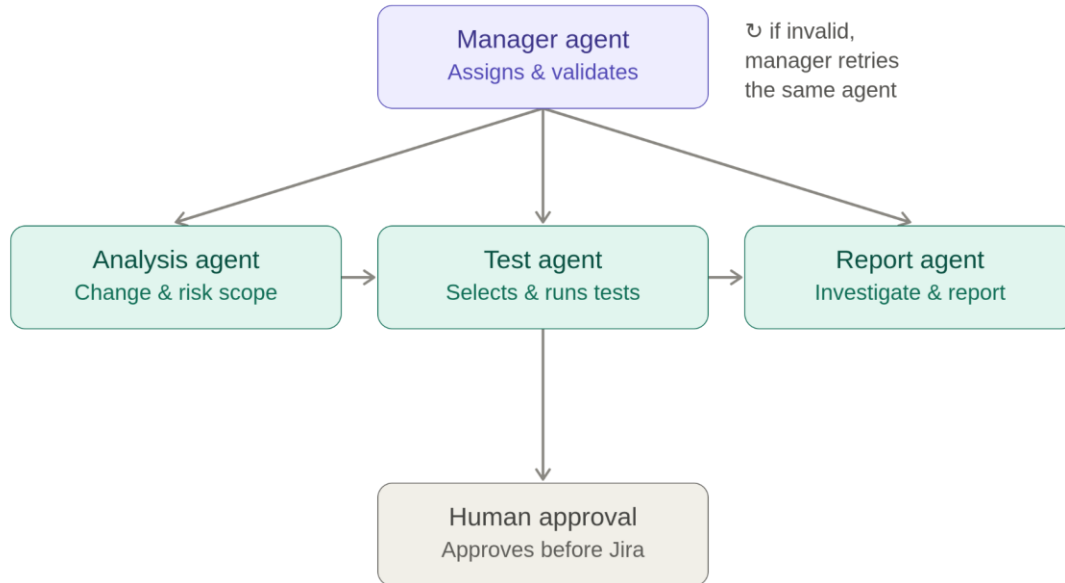
The workflow is built using Google ADK [1], chosen because the process needs to branch, retry failed steps, pause for human approval before filing a ticket, and resume safely if something goes wrong. A plain, one direction chain of prompts cannot support this kind of pausing and retrying, so a graph based framework is used instead. Google ADK's built in session and state service [2] automatically saves the state of a run after every step, so a run can resume safely after a crash without extra code.

A Manager Agent sits above three worker agents and does not carry out testing work itself. It hands each worker its task, checks the returned output against what is expected, and either lets the workflow continue to the next agent or sends the same agent back the task again with feedback on what needs to be fixed. Retries are capped at a small fixed number of attempts, so a stuck agent cannot loop forever without being flagged for human review , this human checkpoint exists specifically to catch a stuck or repeatedly wrong agent before it wastes further time, rather than letting it retry indefinitely. This manager and worker structure follows the supervisor and

orchestrator worker pattern described by both Anthropic [12] and OpenAI [13] as the standard way to keep a multi agent system reliable.

Each worker agent reaches the outside world through a standard integration adapter that lets it call a real tool, such as GitHub or Jira, in a safe and predictable way, instead of the team writing custom connection code for each one.

- Analysis Agent: reads the pull request difference, meaning the exact lines of code that were changed, through GitHub's official integration, and produces a rule based risk label, calculated from the files changed, the module touched, and how often that module has produced defects in the past. If a pull request touches a file or module with no prior defect history, such as a brand new module or a fresh repository, the risk label defaults to Medium rather than Low, so a genuinely new area of the codebase is not mistaken for a proven, low risk one.
- Test Agent: takes the Analysis Agent's output and selects the relevant Playwright user interface tests, run through Playwright's official integration, and the relevant Newman [6] API collection, then runs them against the staging environment. If a selected test targets a module or endpoint that is not yet present on staging, because deployment to staging happens on its own schedule outside this system, the Test Agent marks that test as deployment pending instead of running it, so a feature that has simply not been deployed yet is never reported as a failing test. When a test does run and fails, the Test Agent marks the failure as a real bug, or a temporary error like a timeout that gets retried once, or a deployment pending case as explained above. This way, a temporary error or a missing deployment is never sent to the Report Agent as a real bug.
- Report Agent: correlates the Test Agent's failure evidence, meaning screenshots, trace files, and console and network logs, into a plain language root cause hypothesis, and drafts the Jira ticket from that hypothesis. Before evidence is attached to a ticket, the Report Agent removes common sensitive values such as API keys, tokens, and passwords found in logs or network traces.



A GitHub webhook, created automatically the moment a tester connects their repository, notifies the backend whenever a pull request is opened or updated. This triggers the Analysis Agent right away, but running the actual tests still needs the tester to click Run Now, and filing the ticket still needs explicit approval, so a human always stays in control of the two most consequential steps

5.2 Language Models

Verixa uses two language models, matched to task difficulty rather than a single model for everything. The latest Google Gemini free tier model [10] handles complex reasoning tasks such as risk labelling, failure summaries, and ticket drafting, chosen for its fast response time and generous free daily quota. Groq [11] handles simpler, high volume tasks, such as quick classification calls, and also serves as a fallback when the Gemini quota is close to running out. Running a self hosted model was considered and rejected for this stage, since the hosting cost and ongoing maintenance would outweigh the benefit at this scale, and smaller open weight models reason noticeably worse on structured tool use tasks.

5.3 State and Data

PostgreSQL with the pgvector [7] extension serves as the single data layer for the whole system. It stores the Google ADK session state so a run can resume after a crash, the ordinary structured data such as pull requests, risk scores, test runs, and Jira links, and it is also ready to store vector embeddings for the long term QA memory feature planned for next semester.

5.4 Integrations

Instead of building custom integrations with GitHub, Playwright, and Jira from scratch, Verixa connects to each one through free, officially maintained integrations. This keeps the amount of new integration code small and lets the team focus effort on the orchestration logic that is actually new.

- GitHub: the official GitHub integration [3] handles pull request data, diffs, and repository operations.
- Playwright: the official Microsoft Playwright integration [4] drives the browser through structured accessibility snapshots rather than guessing from screenshots.
- Jira and Confluence: the official Atlassian integration [5] files and updates tickets, with a community maintained fallback option kept in reserve.

Postman collections do not have an official integration available, so the Test Agent calls Newman [6] directly as a command line subprocess for API testing instead.

5.5 Backend and Dashboard

FastAPI [8] exposes the REST and WebSocket endpoints that the web dashboard calls into, so every action goes through one backend, one run object, and one evidence store. The dashboard itself is built with React and TypeScript, kept intentionally small for this semester, covering some screens needed for the deliverables rather than a full product interface.

6. Project Scope

The first semester of Verixa is scoped around one complete and working loop rather than many partial features. Of the ten deliverables in the original idea, seven are built this semester because together they form one working chain from a pull request being opened to a ticket being filed. The remaining four are carried forward to the second semester rather than dropped.

6.1 In Scope for This Semester

- Connecting a GitHub repository and a Jira project through one login, usable from either the dashboard.
- Uploading already written Playwright test files and a Postman collection, which are parsed and added to a Test Library.
- Automatic, rule based risk scoring of a newly opened pull request.
- Running the relevant Playwright and Newman tests with one click or one command.

- Drafting a new test case when the Test Agent finds a coverage gap in the uploaded Playwright or Postman tests.
- Viewing pass and fail results together with screenshots, trace files, and console and network logs.
- Reviewing a failure, approving it, and having a Jira ticket filed automatically with the evidence attached.

6.2 Deferred to Next Semester, Not Dropped

- QA Memory, which recalls semantically similar past bugs using vector embeddings.
- Suggest a Fix, which drafts a candidate fix suggestion from the diagnosed root cause.
- Regression Comparison, which compares two test runs side by side.
- Cross-PR Risk Trend Reporting, which aggregates risk scores across many pull requests over time.

6.3 Out of Scope for the Full Year

- Autonomous exploratory testing, where an agent roams the application on its own looking for bugs.
- Security or load and performance testing.
- Mobile application testing and production environment monitoring.
- Automatically merging a fix pull request without a human review.

6.4 Assumptions

- Tests run against a staging or development environment, not a live production system.
- Uploaded Playwright specs are written in TypeScript; plain JavaScript files are not accepted this semester.
- Test cases already exist as automated Playwright specs or a Postman collection.
- The tester manually approves both running the tests and filing the Jira ticket; nothing reaches Jira without a human clicking approve.

7. Feasibility Study

7.1 Risks Involved

- Language model rate limits: Gemini's free tier is limited to about 1500 requests per day. This is managed by routing overflow and quick classification calls to the Groq hosted Llama 3.3 70B model, which is also free.

- Uncapped retries: if the Manager Agent kept retrying a failing worker agent forever, a run could stall indefinitely. This is managed by capping retries at a small fixed number of attempts before the run is flagged for human review.
- Two person team building within one semester: the risk of spreading effort too thin across too many features is managed by narrowing this semester to seven deliverables that form one complete loop, rather than attempting all ten deliverables from the original idea at once.
- Third party integration availability: since GitHub, Playwright, and Jira integrations depend on externally maintained services, a community maintained fallback is kept ready in case the official Atlassian service has downtime.
- Evidence volume: screenshots, trace files, and logs can grow large over many test runs. This is managed by storing the raw evidence files on the server's own storage, with only their file locations kept in PostgreSQL, so the database itself stays small and fast.
- Staging environment mismatch: since staging deployment stays a separate manual process outside this system's scope, a pull request's code may not yet be present on staging at the moment its tests run. This is managed by the deployment pending check described in section 5.1, so an undeployed feature is never mistaken for a defect.

7.2 Resource Requirement

- A development machine capable of running PostgreSQL, a Python backend, and a Node based frontend locally.
- A PostgreSQL database with the pgvector extension enabled, either running locally or on a small free tier cloud instance.
- A GitHub account with permission to install a GitHub App or OAuth App on a test repository.
- An Atlassian Cloud account with a Jira project available for testing ticket creation.
- Free tier API access to Google Gemini and to Groq, both of which are available without a credit card at the usage levels this project needs.
- Node.js for running Playwright and Newman, and Python for the FastAPI backend and the Google ADK agents.
- A small staging web application, either an existing sample application or one built for demonstration, against which the uploaded tests are run.

8. Solution Application Areas

Verixa is built around a team of smart AI agents that work alongside a QA team, each one handling a specific part of the job so testers do not have to move between five separate tools by hand. Any software team that reviews pull requests on GitHub and tracks defects on Jira can benefit from

this. A small team without a dedicated QA engineer can use it to get consistent, evidence backed defect reports without hiring more testing staff. A larger QA department can use it to cut down the time spent manually correlating screenshots and logs into a ticket, freeing testers to spend more time on exploratory testing and judgement calls that genuinely need a human. Because the human approval gate stays in place before anything reaches Jira, teams keep full control over what actually gets filed, which makes the tool easier to trust and adopt gradually.

9. Tools/Technology

The following tools and technologies are used to build Verixa.

Category	Tool / Technology	Purpose
Agent Orchestration	Google ADK	Builds the branching, retryable, human in the loop agent workflow
Backend API	FastAPI (Python)	Exposes REST and WebSocket endpoints to the dashboard
Frontend Dashboard	React with TypeScript	Web interface for connecting projects, viewing risk, and reviewing evidence
Primary Language Model	Google Gemini/local model, latest free tier model	Risk reasoning, failure summaries, and ticket drafting
Fallback Language Model	Groq (Llama 3.3 70B)	Handles overflow and quick classification calls
Text Processing	sentence-transformers [9]	Helps the system find similar past bugs by meaning, for the future QA memory feature
Database	PostgreSQL with pgvector	Stores the project's data, including what the future QA memory feature will search through.
UI Testing	Playwright	Runs uploaded UI test specs against staging
API Testing	Newman (Postman collection runner)	Runs the uploaded Postman collection against staging

Source Control Integration	GitHub	Reads pull request diffs and repository data
Issue Tracking Integration	Jira	Files and updates Jira tickets with evidence attached

10. Expertise of the Team Members

Arisha Mumtaz and Farheen Imam are both final year Computer Science students at SZABIST University, and have completed the relevant coursework leading into this project, including software engineering, databases, web engineering, and artificial intelligence related courses. Arisha has a particular interest in software quality assurance, while Farheen's focus leans more toward applied artificial intelligence and agentic systems, and together this shared interest across both areas is what motivated the choice of this project. Work is divided so that both members are involved across the agent logic, the backend, and the integrations, rather than splitting strictly along frontend and backend lines, so that both members build a working understanding of the whole system rather than only their own piece of it.

11. Milestones

The table below lists the major tasks and sub tasks planned across the full one year project, split between this semester and the next.

11.1 FYP-I Features

The first semester delivers the one complete loop described in the Methodology and Scope sections above, covering the four agents from the moment a pull request opens to the moment a ticket is filed.

- Account and Project Setup: connecting a GitHub repository and a Jira project through one login triggers automatic webhook provisioning in the background, so a tester never has to configure a webhook by hand or reconnect after every new pull request.
- Test Library Upload: existing Playwright specs and a Postman collection are parsed and shown in a live preview before the tester confirms the import, so mistakes in the uploaded files are caught before they become part of the Test Library rather than after.
- Pull Request Risk Scoring: the Analysis Agent labels a newly opened pull request as Low, Medium, or High based on the files changed, the module touched, and how often that module

has produced defects before, so a tester knows within seconds which pull requests need closer attention.

- **One Click Test Execution:** the Test Agent runs the relevant Playwright user interface tests and Newman API tests together from a single action, so a tester no longer needs to trigger UI and API test runs separately or track them across different tools.
- **Coverage Gap Test Drafting:** whenever the Test Agent finds a part of the change with no matching test in the uploaded specs, it drafts a new test case to cover it, so gaps in the tester's existing suite get filled automatically instead of being discovered later during manual review.
- **Results and Evidence View:** pass and fail results appear together with their screenshots, trace files, and logs in one view, so a tester investigating a failure no longer needs to open several separate tools to piece the evidence together.
- **Manual File to Jira:** the Report Agent drafts a ticket from the failure evidence it has gathered, and a human reviews and approves it before it is filed, so nothing reaches Jira without a tester's final sign off.

11.2 FYP-II Features

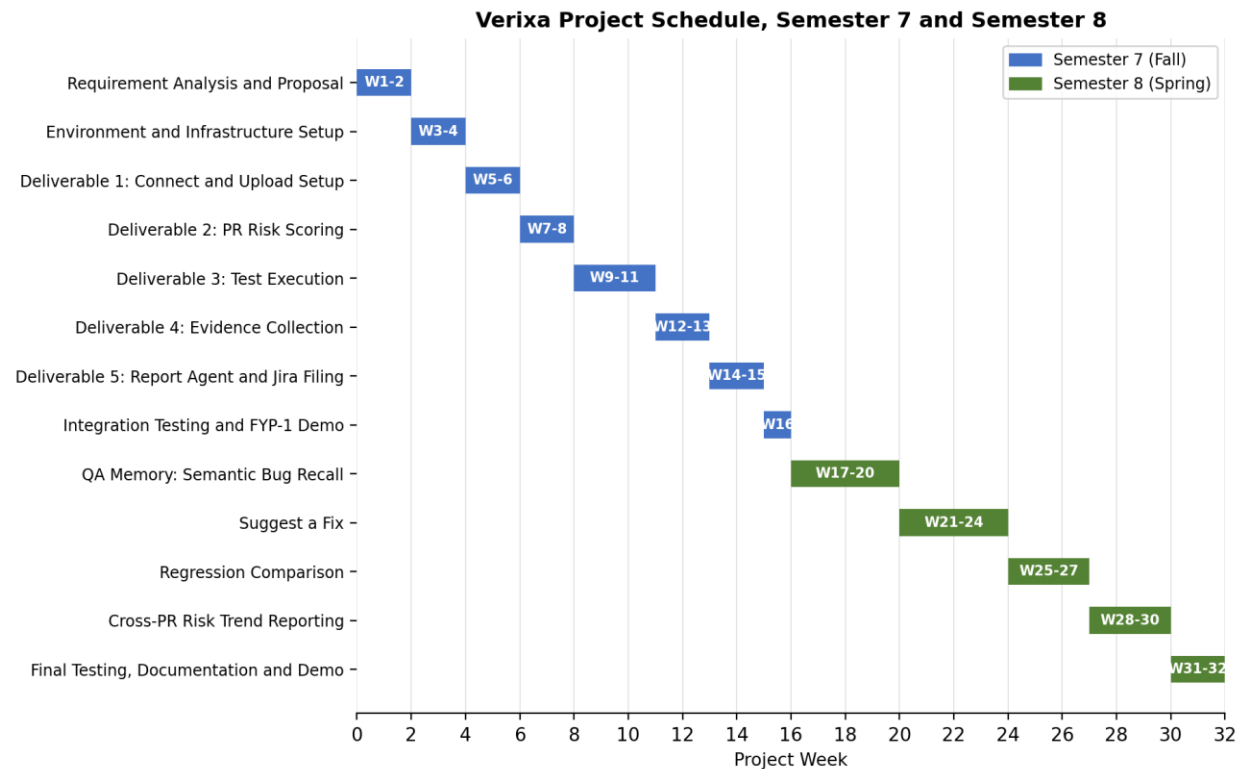
The second semester deepens the same four agent pipeline built in the first semester, rather than introducing new agents or new external infrastructure. Each feature below builds directly on the Analysis Agent, Test Agent, and Report Agent already in place.

- **QA Memory (Semantic Bug Recall):** when a new test failure is investigated, the Report Agent searches vector embeddings, stored using pgvector, of past bug reports and their root cause hypotheses, so it can surface a matching past incident even if the wording of the new error is different. This lets a tester see instantly that a failure looks like a bug seen before, instead of investigating a known pattern from zero.
- **Suggest a Fix:** building on the Report Agent's root cause hypothesis, this feature drafts a possible fix and attaches it to the ticket as a starting point for the developer. It is always reviewed by a human and never applied automatically, since it is meant to speed up the fix, not replace the developer's judgement.
- **Regression Comparison:** testers can select two test runs, for example before and after a fix, or across two releases, and see a side by side comparison of which tests newly failed, which newly passed, and which stayed flaky. This turns raw run history into an actionable comparison, making it easy to confirm whether a change actually fixed something or introduced a new problem, without manually cross referencing logs.

- Cross-PR Risk Trend Reporting: a management facing view that aggregates the risk scores already assigned to every pull request over time. It surfaces patterns such as which modules repeatedly produce high risk changes, and how risk and defect trends shift from week to week, giving team leads visibility into recurring problem areas without opening individual pull requests one by one.

12. Project Schedule

The chart below shows a preliminary one year schedule for Verixa, covering both this semester and the next. Weeks 1 to 16 cover the 7th semester and the seven deliverables described above. Weeks 17 to 32 cover the 8th semester and the four features that extend the same agent pipeline. A closer, semester by semester view of this same schedule is given alongside the Work Breakdown Structure in Section 13.

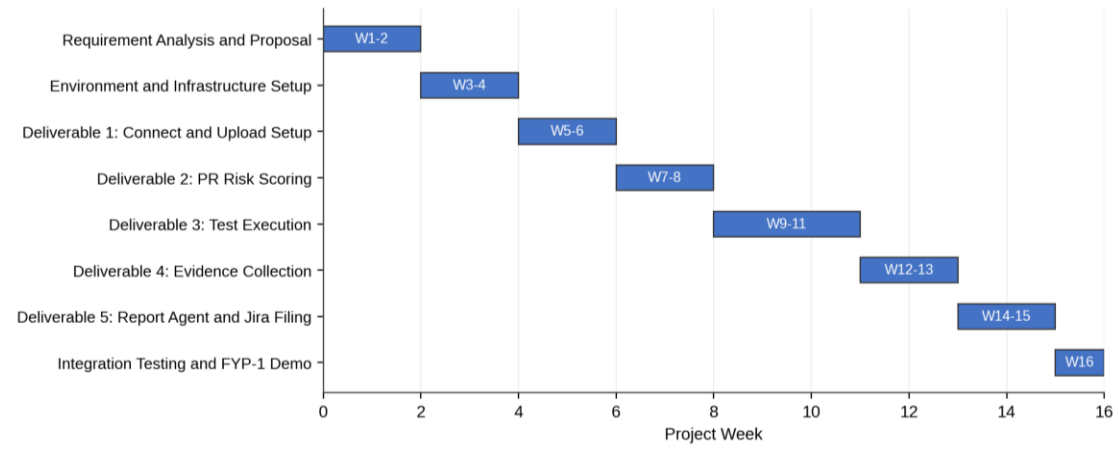


13. Work Breakdown Structure

The work for the two semesters is broken down below into phases. Each phase spans one or more weeks, and each task within a phase follows the usual software development steps of design, build, and test.

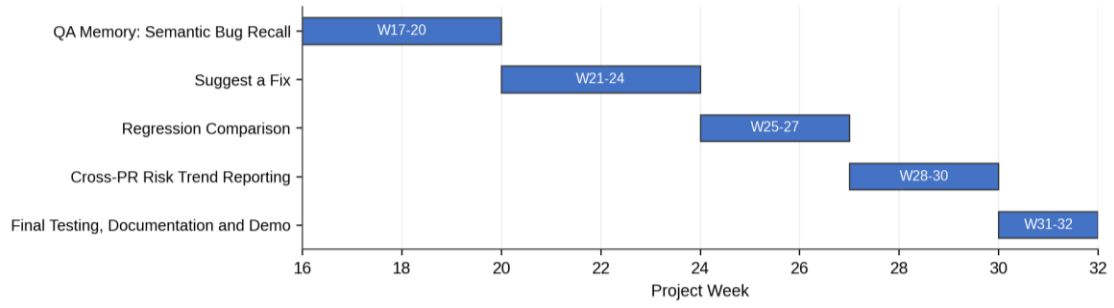
13.1 Semester 1 (Fall): FYP-1 Work Breakdown

Week	Phase	Tasks	Milestone
1 – 2	Requirement Analysis and Proposal	Finalise scope, write and submit the FYP-1 proposal, confirm the seven deliverables for this semester	Approved proposal document
3 – 4	Environment and Infrastructure Setup	Set up PostgreSQL with pgvector, scaffold the FastAPI backend, register the GitHub OAuth App and Jira project	Working development environment
5 – 6	Deliverable 1: Connect and Upload Setup	Build GitHub and Jira connection flow, automatic webhook setup, and the spec/collection upload widget with live preview	Account and Project Setup complete
7 – 8	Deliverable 2: PR Risk Scoring	Build the Analysis Agent, connect it to GitHub, implement the rule based risk label	PR and Risk Dashboard working
9 – 11	Deliverable 3: Test Execution	Build the Test Agent, connect it to Playwright and Newman, wire up one click test execution	One click Test Run working
12 – 13	Deliverable 4: Evidence Collection	Integrate Playwright's Trace Viewer and logs, build the results and evidence screen	Results and Evidence View complete
14 – 15	Deliverable 5: Report Agent and Jira Filing	Build the Report Agent, connect it to Jira, implement the human approval gate before filing	Manual File to Jira working
16	Integration Testing and FYP-1 Demo	End to end test of the full loop through the dashboard, prepare documentation and the live demo	FYP-1 demo ready



13.2 Semester 2 (Spring): FYP-2 Work Breakdown

Week	Phase	Tasks	Milestone
17 – 20	QA Memory: Semantic Bug Recall	Store past defects in a way the system can search by meaning, not just exact words, so when a new bug is found, the Report Agent can show similar bugs found before	QA Memory feature working
21 – 24	Suggest a Fix	Extend the Report Agent to draft a candidate fix from the root cause and the code diff, attach it to the ticket as evidence	Suggest a Fix feature working
25 – 27	Regression Comparison	Build the run selection and side by side comparison view for two test runs	Regression Comparison feature working
28 – 30	Cross-PR Risk Trend Reporting	Collect the risk scores from every pull request over time, and build a dashboard view that shows which modules are repeatedly flagged as high risk and whether the project's overall risk is trending up or down, giving a QA lead a project wide view instead of only one PR at a time	Risk Trend Reporting feature working
31 – 32	Final Testing, Documentation and Demo	Full system regression test, finalise thesis documentation, prepare the final demo	FYP-2 final demo ready



14. References

The following literature and web references were used while researching the tools and architectural patterns applied in this project. Each reference is numbered and cited in the text above using square brackets where relevant.

- [1] Google. "Agent Development Kit Overview." ADK Docs by Google. Accessed August 26, 2026. <https://google.github.io/adk-docs/get-started/about/>
- [2] Google. "Session: Tracking Individual Conversations." ADK Docs by Google. Accessed August 26, 2026. <https://google.github.io/adk-docs/sessions/session/>
- [3] GitHub. "GitHub REST API Documentation." GitHub Docs. Accessed August 26, 2026. <https://docs.github.com/en/rest>
- [4] Microsoft. "Playwright Documentation." Playwright Docs. Accessed August 26, 2026. <https://playwright.dev/docs/intro>
- [5] Atlassian. "Jira REST API Documentation." Atlassian Developer. Accessed August 26, 2026. <https://developer.atlassian.com/cloud/jira/platform/rest/v3/intro/>
- [6] Postman. "postmanlabs/newman: Command Line Collection Runner for Postman." GitHub. Accessed August 26, 2026. <https://github.com/postmanlabs/newman>
- [7] pgvector. "pgvector: Open Source Vector Similarity Search for Postgres." GitHub. Accessed August 26, 2026. <https://github.com/pgvector/pgvector>
- [8] Tiangolo, Sebastian. "FastAPI." Official Documentation. Accessed August 26, 2026. <https://fastapi.tiangolo.com/>
- [9] Hugging Face. "Sentence Transformers Documentation." Accessed August 26, 2026. <https://sbert.net/>
- [10] Google. "Gemini Developer API Pricing." Gemini API Docs by Google. Accessed August 26, 2026. <https://ai.google.dev/gemini-api/docs/pricing>
- [11] Groq. "Groq is the Premier Neocloud for Fast Inference." Accessed August 26, 2026. <https://groq.com/>
- [12] Anthropic. "Building Effective Agents." Anthropic Engineering. Accessed August 26, 2026. <https://www.anthropic.com/engineering/building-effective-agents>
- [13] OpenAI. "A Practical Guide to Building Agents." Accessed August 26, 2026. <https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf>